PRE-ENTREGA 5:

Documento de arquitectura de datos RDS vs. DynamoDB

Proyecto: Startup de educación en línea sobre AWS

Alumno: Germán Fondato.

Año: 2026.

Supuestos de partida: La arquitectura de los módulos anteriores; servidor web en EC2 (t3.micro) dentro de un Auto Scaling Group detrás de un Application Load Balancer, en una región (São Paulo, sa-east-1) y dos zonas de disponibilidad, más una función Lambda con trigger SNS que envía el correo de bienvenida al registrarse. Este documento agrega la capa de datos.

1- Cuadro comparativo de requisitos

Se revisaron los datos que maneja la plataforma y qué características exige cada uno.

Dato	Qué guarda	Características	Requisito Clave	Encaje
Perfiles de estudiantes	Nombre, correo, fecha de registro	Estructura fija. Se relaciona con inscripciones	Integridad (correo único)	RDS
Catálogo de cursos	Cursos, módulos, instructores, precios	Relaciones jerárquicas. Muchas lecturas, pocas escrituras	JOINs y consultas flexibles	RDS
Inscripciones y pagos	Quién cursa qué y estado del pago	Transacciones. No puede perderse ni duplicarse	Consistencia ACID	RDS
Progreso del estudiante	Lecciones completadas y calificaciones	Ligado a inscripciones. Se consulta en reportes	Consultas agregadas (SQL)	RDS
Reportes de negocio	Alumnos activos por curso, ingresos	Consultas nuevas y variadas	SQL ad hoc	RDS

Volumen estimado: De miles a cientos de miles de filas, con picos al abrir cada curso (decenas a cientos de operarios por segundo). No se esperan millones de micro-operaciones por segundo, que serían la señal para inclinarse hacia DynamoDB.

2- RDS vs. DynamoDB

Comparación de ambos servicios para el caso de esta plataforma.

Criterio	Amazon RDS	Amazon DynamoDB	Mejor para este caso
Modelo de datos	Relacional. Tablas con equema fijo	NoSQL clave-valor y documentos. Esquema flexible.	RDS: Los datos mantienen una estructura estable
Relaciones y JOINs	Soporta JOINs y claves foráneas	No soporta JOINs. Hay que duplicar datos o resolverlo en la aplicación	RDS
Consultas	SQL: consultas ad hoc y reportes	Acceso por clave de partición y ordenación. Consultas nuevas piden índices	RDS
Transacciones y consistencia	Transacciones ACID completas	Transacciones con límites. Lecturas eventualmente consistentes por defecto	RDS
Escalabilidad	Vertical (instancia mayor) y réplicas de lectura. Escrituras más limitadas	Horizontal y automática. Prácticamente sin límite.	DynamoDB (no es crítico acá)
Alta disponibilidad	Multi-AZ con failover automático	Replicado en varias AZ por defecto	Igual para ambos
Administración	Gestionado (parches, backups), pero se elige instancia y almacenamiento	Serverless: no hay servidores que dimensionar	DynamoDB
Modelo de costo	Por hora de instancia más almacenamiento. Previsible con tráfico estable	Por capacidad o por solicitud. Ideal con tráfico impredecible	RDS: Tráfico estable y reportes
Seguridad de red	Vive en subredes privadas de la VPC, protegida por Security Groups	Se accede por API con IAM. Se puede restringir con un endpoint de VPC	RDS: Control por Security Groups

3- Justificación técnica

Elegí Amazon RDS con MySQL porque los datos de la plataforma son relacionales: Un estudiante se inscribe en varios cursos, cada curso tiene módulos e instructores, y cada inscripción tiene un pago asociado. El equipo necesita preguntas que crucen esas entidades, como cuántos alumnos activos tiene cada curso o qué inscripciones siguen impagas. Con SQL eso se resuelve con un JOIN, mientras que DynamoDB no soporta JOINs y obligaría a duplicar datos o resolverlo en el código de la aplicación.

El segundo punto que encontré como beneficio es la consistencia transaccional. Registrar una inscripción y su pago debe ocurrir completo o no ocurrir, y RDS ofrece transacciones ACID de manera nativa. Así los reportes de ingresos son confiables y se evita que alumnos que ya hayan pagado no tengan acceso al curso.

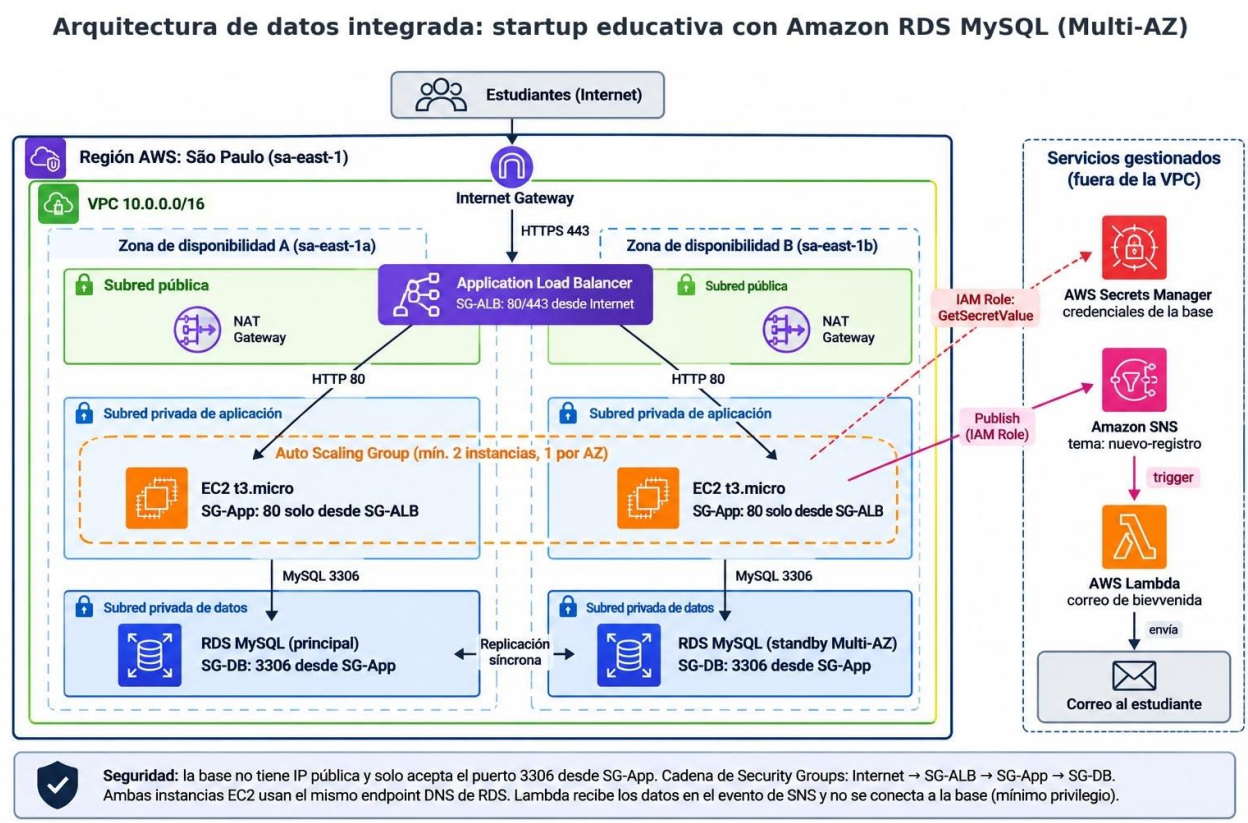
Tercero, la escritura de los datos es estable. Alumnos, cursos e inscripciones no cambian de forma, por lo que el esquema fijo es una ventaja, ya que la base valida por nosotros.

Sobre el volumen, la plataforma tendrá picos al abrir cada curso, pero no millones de operaciones por segundo, lo cual si justificaría utilizar DynamoDB. Una instancia db.t3.micro con Multi-AZ cubre la demanda con un costo mensual previsible. Descarté Aurora porque, para un proyecto inicial, RDS MySQL cumple el requisito con menos gasto.

Además de lo mencionado anteriormente, RDS es un servicio gestionado: AWS aplica parches, hace backups automáticos y realiza el failover. La base vive en subredes privadas y solo las instancias EC2 pueden conectarse.

Como contrapartida, escalar arquitecturas es más limitado que DynamoDB, pero si más adelante aparece un dato de altísimo volumen, como eventos de progreso de video, se podría sumar DynamoDB solo para casos como este.

4- Diagrama de integración actualizado.



Flujo de comunicación entre cómputo y datos

1- El estudiante entra por HTTPS al Application Load Balancer (ALB), a través de Internet Gateway.
2- El ALB reparte el tráfico por HTTP hacia las instancias EC2 del Auto Scaling Group (una por zona de disponibilidad), ubicadas en subredes privadas.
3- La aplicación obtiene las credenciales desde Secrets Managers con el IAM Role de la instancia y se conecta con el endpoint DNS de RDS por el puerto 3306.
4- Al registrarse un estudiante, la aplicación lo guarda en RDS y publica un evento en el tema de SNS.
5- SNS dispara la función Lambda, que envía el correo de bienvenida con los datos del evento. La Lambda no se conecta a la base de datos.
6- Si falla la zona de disponibilidad de la base principal, RDS promueve el standby (Multi-AZ) y el endpoint DNS no cambia.

Configuración propuesta de la base de datos

Criterio	Amazon RDS
Motor	Amazon RDS for MySQL (no Aurora)
Clase de instancia	Db.t3.micro (ajustable según demanda)
Alta disponibilidad	Multi-AZ: Principal en sa-east-1a y standby en sa-east-1b
Almacenamiento	SSD gp3 de 20 GiB con escalado automático
Backups	Automáticos, retención de 7 días
Acceso Público	No (Publicly accessible = No), subredes privadas de datos
Cifrado	En reposo (KMS) y en tránsito (TLS)

5- Seguridad del acceso de cómputo a datos

Medida	Configuración	Qué previene
Subred privada de datos	RDS sin IP pública y sin ruta al Internet Gateway	Acceso directo desde internet
Security Groups encadenados	SG-ALB: 80/443 desde Internet. SG-App: 80 solo desde SG-ALB. SG-DB: 3306 solo desde SG-App	Que cualquier otro origen llegue a la base
IAM Role de la instancia EC2	Permisos mínimos: leer el secreto de la base (secretsmanager:GetSecretValue) y publicar en el tema SNS	Credenciales en el código y permisos excesivos
AWS Secrets Manager	Usuario y contraseña de la base guardados allí, con rotación habilitada	Contraseñas en texto plano
Usuario de base con mínimo privilegio	La aplicación usa un usuario limitado a su esquema, no el administrador	Daño mayor ante un fallo de la aplicación
Cifrado y backups	KMS en reposo, TLS en tránsito, backups automáticos Multi-AZ	Exposición y pérdida de datos
IAM Role de Lambda	Rol propio con permiso solo para enviar correo y escribir logs. Sin acceso a la base	Que la Lambda sea una puerta a los datos

Errores comunes evitados:

- No elegí lo más potente: RDS MySQL en lugar de Aurora, porque el requerimiento no lo exige.
- La base de datos no está abierta a Internet: solo acepta tráfico desde las instancias de aplicación.
- No mezclé conceptos: DynamoDB no soporte JOINs, por eso no se adapta a las consultas relacionales de esta plataforma.